

Модуль 12. Масштабирование, ребалансировка и идиempотентность

12.1. Введение: кульминация курса

Мы прошли путь от первого контейнера `hello-world` до асинхронного API, публикующего события в Kafka. Мы знаем, как упаковать код в Docker, как собрать многоэтапный образ, как связать сервисы в Compose, как работают брокеры, как читать потоки и ставить задачи в очередь.

Но вся эта инфраструктура бессмысленна, если она ломается под нагрузкой или при сбоях. Реальный production — это не идеальный мир, где один воркер работает вечно. Это мир, где:

- Нагрузка растёт, и нужно добавлять серверы.
- Серверы падают из-за перегрева, нехватки памяти или сетевых сбоев.
- Сообщения иногда обрабатываются дважды.
- «Битые» данные заставляют задачи падать в бесконечном цикле.

Этот модуль учит трём финальным навыкам:

1. **Масштабирование** — добавлять воркеров без остановки системы.
2. **Ребалансировка** — как система переживает падение и появление новых воркеров.
3. **Идиempотентность** — как гарантировать, что повторная обработка не сломает данные.

12.2. Масштабирование воркеров: от одного к армии

12.2.1. Что значит «масштабировать»

Масштабирование — это увеличение вычислительной мощности системы путём добавления новых экземпляров (контейнеров, процессов, серверов).

Аналогия: кухня ресторана

В обеденный час поток заказов вырос в 5 раз. У вас два варианта:

- **Вертикальное масштабирование:** Нанять супер-повара, который готовит в 5 раз быстрее. Но быстрее определённого предела невозможно — руки всё равно две.
- **Горизонтальное масштабирование:** Нанять 5 обычных поваров. Каждый берёт заказы из общей стопки. Это дешевле, надёжнее и бесконечно масштабируется (пока хватает места на кухне).

В контейнерном мире мы используем **горизонтальное масштабирование**: запускаем больше контейнеров с воркерами.

12.2.2. Масштабирование Celery Workers

Когда вы запускаете:

```
In [1]: celery -A celery_app worker --loglevel=info
```

Celery создаёт **пул процессов** (по умолчанию по числу ядер CPU). Но это один инстанс на одной машине.

Чтобы масштабировать, вы просто запускаете **ещё один** такой же процесс (или контейнер) на той же или другой машине. Оба воркера подключаются к одному Redis-брокеру и делят между собой задачи из очереди.

Практика:

Откройте **три** терминала.

Терминал 1 — запускаем первый воркер:

```
In [2]: cd ~/docker-module8
source venv/bin/activate
celery -A celery_app worker --loglevel=info -n worker1@%h
```

Флаг `-n worker1@%h` задаёт уникальное имя воркера.

Терминал 2 — запускаем второй воркер:

```
In [3]: celery -A celery_app worker --loglevel=info -n worker2@%h
```

Терминал 3 — отправляем много задач:

```
In [4]: # send_many.py
from tasks import train_model

for i in range(10):
    train_model.delay(f"dataset_{i}.csv", "random_forest")
    print(f"Отправлена задача {i}")
```

```
In [5]: python send_many.py
```

Наблюдайте: Задачи распределяются между `worker1` и `worker2`. Один не перегружается, пока другой бездельничает. Redis List гарантирует, что одна задача попадает только одному воркеру.

12.2.3. Масштабирование Kafka Consumers

В Kafka масштабирование работает через **Consumer Groups** и **партиции**.

Золотое правило:

Максимальное количество параллельных консьюмеров в группе равно количеству партиций топика.

Если у топика `user.events` **3 партиции**, то в группе `analytics` может работать **максимум 3 консьюмера** одновременно. Четвёртый будет простаивать.

Практика:

Запустите три консольных консьюмера из одной группы (как в Модуле 10), затем отправьте 6 сообщений через Producer. Вы увидите, что Kafka назначила каждому консьюмеру по одной партиции.

Теперь запустите **четвёртого** консьюмера в той же группе:

```
In [6]: docker exec -it kafka_broker \
        kafka-console-consumer.sh \
        --topic user.events \
        --group analytics \
        --bootstrap-server localhost:9092
```

Проверьте распределение:

```
In [7]: docker exec -it kafka_broker \
        kafka-consumer-groups.sh \
        --bootstrap-server localhost:9092 \
        --describe \
        --group analytics
```

Вы увидите, что один консьюмер не получил партицию (столбец `CURRENT-OFFSET` будет пуст или `-`). Он «в резерве».

12.3. Ребалансировка (Rebalance): перераспределение при сбоях

12.3.1. Что такое ребалансировка

Ребалансировка — это процесс, при котором Kafka перераспределяет партиции между живыми членами Consumer Group.

Аналогия: перегруппировка в спортивной команде

Вы играете в футбол 5 на 5. У каждого своя зона на поле. Внезапно один игрок травмировался и выбыл. Команда останавливает игру, быстро перераспределяет зоны: теперь 4 игрока покрывают то же поле. Когда игрок возвращается — зоны снова пересматриваются.

Когда происходит ребалансировка:

1. **Новый Consumer присоединился к группе.** Нужно отдать ему часть партиций.
2. **Consumer покинул группу** (упал, был убит, потерял соединение).
3. **Consumer «завис»** — не отправил heartbeat вовремя. Kafka считает его мёртвым.
4. **Администратор изменил топик** — добавил новые партиции.

12.3.2. Как это работает технически

Consumer постоянно отправляет **heartbeat** (сигнал «я жив») специальному брокеру — **Group Coordinator**. Если heartbeat не пришёл в течение `session.timeout.ms` (по умолчанию 45 секунд), Coordinator объявляет Consumer мёртвым и инициирует ребалансировку.

Во время ребалансировки:

- Все Consumer'ы группы временно **останавливают чтение**.
- Group Coordinator пересчитывает назначения.
- Consumer'ы получают новые партиции.
- Чтение возобновляется.

Проблема: Во время ребалансировки система не обрабатывает сообщения. Если ребалансировки частые (например, Consumer'ы постоянно перезапускаются из-за нестабильной сети), это создаёт «простой» (downtime).

12.3.3. Eager vs Cooperative Rebalancing

Существуют два протокола ребалансировки:

Eager (жадный, старый):

- Все Consumer'ы **сбрасывают все партиции**.
- Затем Coordinator перераспределяет всё с нуля.

- Проблема: даже если изменился только один Consumer, все остальные на мгновение теряют свои партиции и прерывают обработку.

Cooperative (кооперативный, современный):

- Перераспределяются **только те партиции**, которые необходимо переместить.
- Остальные Consumer'ы продолжают работу без прерываний.
- Это стратегия `CooperativeStickyAssignor` в Kafka.

В `aiokafka` cooperative rebalancing включается через параметр `partition_assignment_strategy`.

12.3.4. Практика: увидеть ребалансировку вживую

Шаг 1. Запустите два Python-консьюмера из одной группы в двух терминалах.

Создайте `consumer_rebalance_demo.py` :

```
In [8]: import asyncio
import json
import signal
import sys
from aiokafka import AIOKafkaConsumer

KAFKA_BOOTSTRAP_SERVERS = "localhost:9092"
TOPIC_NAME = "rebalance.test"
GROUP_ID = "rebalance_demo_group"

async def consume(consumer_id: str):
    consumer = AIOKafkaConsumer(
        TOPIC_NAME,
        bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
        group_id=GROUP_ID,
        auto_offset_reset="earliest",
        value_deserializer=lambda v: json.loads(v.decode("utf-8")),
    )

    await consumer.start()
    print(f"[{consumer_id}] Старт. Назначенные партиции: {consumer.assignment()}")

    try:
        async for msg in consumer:
            print(f"[{consumer_id}] Партия {msg.partition}, offset {msg.offset}: {msg.value}")
```

```

except asyncio.CancelledError:
    print(f"\n[{consumer_id}] Остановка...")
finally:
    await consumer.stop()
    print(f"[{consumer_id}] Отключен.")

if __name__ == "__main__":
    consumer_id = sys.argv[1] if len(sys.argv) > 1 else "consumer_1"

    loop = asyncio.new_event_loop()
    asyncio.set_event_loop(loop)

    task = loop.create_task(consume(consumer_id))

    # Корректная остановка по Ctrl+C
    for sig in (signal.SIGINT, signal.SIGTERM):
        loop.add_signal_handler(sig, task.cancel)

    try:
        loop.run_until_complete(task)
    finally:
        loop.close()

```

Шаг 2. Запустите первого:

In [9]: `python consumer_rebalance_demo.py alpha`

Шаг 3. Запустите второго в другом терминале:

In [10]: `python consumer_rebalance_demo.py beta`

Шаг 4. Наблюдайте за логами. Когда `beta` подключился, `alpha` мог на мгновение потерять партиции и получить их заново (в зависимости от версии Kafka и клиента).

Шаг 5. Убейте `beta` (`Ctrl+C`). Наблюдайте, как `alpha` подхватывает освободившиеся партиции.

12.4. Паттерн Идемпотентности: защита от дублей

12.4.1. Почему сообщения дублируются

Мы уже знаем, что Kafka (и Celery с Redis) гарантирует **at-least-once** доставку. Сообщение гарантированно не потеряется, но может быть обработано дважды.

Сценарии дублирования:

1. Consumer обработал сообщение, но упал до commit.

Kafka не получила подтверждение. При перезапуске Consumer получит то же сообщение снова.

2. Ребалансировка во время обработки.

Consumer А читал сообщение из партиции 0. Произошла ребалансировка, партиция 0 перешла к Consumer Б. Consumer Б начинает читать с последнего закоммиченного offset'a — и получает сообщение, которое А уже обработал, но не успел закомитить.

3. Producer отправил сообщение, но не получил ack.

Producer повторил отправку. В Kafka оказались два одинаковых сообщения.

Аналогия: двойное списание денег

Вы платите за кофе картой. Терминал говорит «Ошибка связи». Вы платите ещё раз. На самом деле первый платёж прошёл, просто терминал не получил подтверждение. Теперь с вашего счёта списалось дважды. Это катастрофа.

12.4.2. Что такое идемпотентность

Идемпотентная операция — это операция, которая при повторном выполнении **не меняет результат**.

Математическая аналогия:

- $f(x) = x + 1$ — **не идемпотентна**. $f(5) = 6$, $f(f(5)) = 7$. Повторный вызов меняет результат.
- $f(x) = \text{abs}(x)$ — **идемпотентна**. $f(-5) = 5$, $f(f(-5)) = 5$. Повторный вызов не меняет результат.

Бытовая аналогия: выключатель света

Вы нажимаете выключатель — лампа загорается. Нажимаете ещё раз — лампа всё ещё горит (или всё ещё не горит, если она уже была включена). Состояние не меняется от повторного нажатия. Выключатель — идемпотентен (в идеальном мире без диммеров).

Противоположность: кнопка «Добавить в корзину»

Если вы нажмёте её дважды — в корзине окажется два товара. Это **не идемпотентно**.

12.4.3. Как реализовать идемпотентность

Способ 1: Уникальный идентификатор сообщения (message_id)

Каждое сообщение получает UUID. Consumer перед обработкой проверяет: «Обрабатывал ли я уже сообщение с таким ID?»

Где хранить историю:

- **Redis:** `SET message:uuid:abc123 "processed" EX 86400` (с TTL 24 часа). Быстро, но память дорога.
- **PostgreSQL:** таблица `processed_messages(id VARCHAR PRIMARY KEY, processed_at TIMESTAMP)`. Надёжно и дешево, но медленнее.

Способ 2: Идемпотентность на уровне бизнес-логики

Вместо «создать заказ» делайте «создать заказ с ID=abc123, если его ещё нет». База данных благодаря `PRIMARY KEY` или `UNIQUE` отклонит дубль.

Способ 3: Состояние «ожидание / выполнено»

Вместо мгновенного выполнения:

1. Записать в БД: «Задача abc123 — статус processing».
2. Выполнить работу.
3. Обновить на «completed».

Если придёт дубль — БД скажет: «Такой ID уже есть, статус completed, пропускаем».

12.5. Практика: идемпотентный Kafka Consumer

Создайте `idempotent_consumer.py` :

```
In [11]: import asyncio
import json
import uuid
from datetime import datetime
from aiokafka import AIOKafkaConsumer, AIOKafkaProducer
import aioredis # или просто redis-asyncio: pip install redis

KAFKA_BOOTSTRAP_SERVERS = "localhost:9092"
TOPIC_NAME = "orders.to_process"
REDIS_URL = "redis://localhost:6379/1" # используем БД 1, чтобы не мешать Celery
```



```

async def is_processed(redis, message_id: str) -> bool:
    """Проверяем, обрабатывали ли уже это сообщение."""
    exists = await redis.get(f"msg:{message_id}")
    return exists is not None

async def mark_processed(redis, message_id: str, ttl: int = 86400):
    """Помечаем сообщение как обработанное с TTL."""
    await redis.set(f"msg:{message_id}", "1", ex=ttl)

async def process_order(order_data: dict) -> dict:
    """Имитация бизнес-логики: создание заказа."""
    await asyncio.sleep(1) # имитация работы
    return {
        "order_id": order_data.get("order_id"),
        "status": "created",
        "total": order_data.get("amount", 0) * 1.2,
        "processed_at": datetime.utcnow().isoformat()
    }

async def run_consumer():
    import redis.asyncio as aioredis

    # Подключаемся к Redis для дедупликации
    redis = aioredis.from_url(REDIS_URL, decode_responses=True)

    consumer = AIOKafkaConsumer(
        TOPIC_NAME,
        bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
        group_id="order_processors",
        auto_offset_reset="earliest",
        value_deserializer=lambda v: json.loads(v.decode("utf-8")),
        enable_auto_commit=False, # ручной контроль
    )

    await consumer.start()
    print("[CONSUMER] Запущен. Ожидая заказы...")

    try:
        async for msg in consumer:
            message_id = msg.value.get("message_id")

            if not message_id:

```

```

        print(f"[WARNING] Сообщение без message_id! Offset {msg.offset}")
        await consumer.commit() # пропускаем, иначе застрянем
        continue

# --- ПРОВЕРКА ИДЕМПОТЕНТНОСТИ ---
if await is_processed(redis, message_id):
    print(f"[SKIP] Сообщение {message_id} уже обработано. Пропуска
ю.")

    await consumer.commit()
    continue

print(f"[PROCESS] Обработка заказа {message_id}...")

try:
    result = await process_order(msg.value)
    print(f"[SUCCESS] Заказ обработан: {result}")

    # Помечаем как обработанное ТОЛЬКО после успеха
    await mark_processed(redis, message_id)

    # И только теперь говорим Kafka: можно двигаться дальше
    await consumer.commit()

except Exception as e:
    print(f"[ERROR] Ошибка обработки {message_id}: {e}")
    # Не делаем commit! Kafka переотправит сообщение.
    # Но благодаря Redis, повторная обработка будет пропущена,
    # если мы уже успели записать в Redis (смотрите комментарий ниже).

except asyncio.CancelledError:
    print("[CONSUMER] Остановка...")
finally:
    await consumer.stop()
    await redis.close()

if __name__ == "__main__":
    asyncio.run(run_consumer())

```

Важный нюанс: В коде выше есть тонкий момент. Если `process_order` упал **после** `mark_processed`, но **до** `commit`, то при повторной доставке сообщение будет пропущено (`is_processed` вернёт `True`), но в бизнес-системе заказ может быть не создан. В production `mark_processed` и `process_order` нужно объединить в одну транзакцию, или проверять результат в БД, а не просто факт обработки в Redis.

Улучшенная версия: Проверьте не «обрабатывали ли», а «существует ли результат в БД». Если заказ с таким `message_id` уже есть в PostgreSQL — пропускаем.

12.6. Dead Letter Queue (DLQ): кладбище битых сообщений

12.6.1. Проблема: бесконечный retry

Представьте, что в очередь попало сообщение с повреждёнными данными. Consumer пытается его обработать — падает. Kafka переотправляет. Consumer снова падает. И так до бесконечности. Это сообщение «отравляет» очередь.

Аналогия: На конвейере по сортировке посылок попала коробка без адреса. Робот не может её обработать. Если робот будет пытаться снова и снова — весь конвейер встанет.

12.6.2. Решение: DLQ

Dead Letter Queue (очередь недоставленных) — это отдельный топик (или очередь), куда отправляются сообщения, которые не удалось обработать после N попыток.

Алгоритм:

1. Consumer получает сообщение.
2. При ошибке — не коммитит offset.
3. Счётчик попыток увеличивается (хранится в заголовке сообщения или Redis).
4. Если попыток ≥ 3 — отправляем сообщение в топик `orders.to_process.dlq`.
5. Основной Consumer коммитит offset (чтобы не застрять).
6. Отдельный сервис (или человек) разбирает DLQ вручную.

12.6.3. Практика: простая DLQ в Python

```
In [12]: MAX_RETRIES = 3
DLQ_TOPIC = "orders.to_process.dlq"

async def run_consumer_with_dlq():
    # ... подключение ...

    async for msg in consumer:
        message_id = msg.value.get("message_id")
        retry_count = msg.value.get("retry_count", 0)

        try:
            await process_order(msg.value)
```

```

        await consumer.commit()

    except Exception as e:
        retry_count += 1

    if retry_count >= MAX_RETRIES:
        # Отправляем в DLQ
        await producer.send(DLQ_TOPIC, {
            **msg.value,
            "retry_count": retry_count,
            "last_error": str(e),
            "failed_at": datetime.utcnow().isoformat()
        })
        print(f"[DLQ] Сообщение {message_id} отправлено в DLQ.")
        await consumer.commit()
    else:
        # Повторим позже — не коммитим offset
        print(f"[RETRY] Сообщение {message_id}, попытка {retry_count}")
        # В Kafka можно использовать pause/resume или просто не коммитить
        # В реальности здесь нужна более сложная логика с задержкой

```

12.7. Практика: полноценный устойчивый Consumer

Создайте `robust_consumer.py` — финальный пример, собирающий всё вместе:

```

In [13]: import asyncio
import json
import logging
import signal

from datetime import datetime
from aiokafka import AIOKafkaConsumer, AIOKafkaProducer
import redis.asyncio as aioredis

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

KAFKA_BOOTSTRAP_SERVERS = "localhost:9092"
TOPIC = "ml.jobs"
DLQ_TOPIC = "ml.jobs.dlq"
GROUP_ID = "robust_ml_workers"
REDIS_URL = "redis://localhost:6379/2"
MAX_RETRIES = 3

class RobustConsumer:

```

```

def __init__(self):
    self.consumer = None
    self.producer = None
    self.redis = None
    self._running = True

async def start(self):
    self.redis = aioredis.from_url(REDIS_URL, decode_responses=True)

    self.producer = AIOKafkaProducer(
        bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
        value_serializer=lambda v: json.dumps(v).encode("utf-8"),
    )
    await self.producer.start()

    self.consumer = AIOKafkaConsumer(
        TOPIC,
        bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
        group_id=GROUP_ID,
        auto_offset_reset="earliest",
        value_deserializer=lambda v: json.loads(v.decode("utf-8")),
        enable_auto_commit=False,
        max_poll_interval_ms=300000, # 5 минут на обработку одного батча
    )
    await self.consumer.start()
    logger.info("Consumer и Producer запущены.")

async def stop(self):
    self._running = False
    if self.consumer:
        await self.consumer.stop()
    if self.producer:
        await self.producer.stop()
    if self.redis:
        await self.redis.close()
    logger.info("Остановлено.")

async def is_duplicate(self, msg_id: str) -> bool:
    return await self.redis.exists(f"processed:{msg_id}")

async def mark_processed(self, msg_id: str):
    await self.redis.set(f"processed:{msg_id}", datetime.utcnow().isoformat
    (), ex=86400*7)

async def process(self, data: dict) -> dict:
    """Имитация долгой ML-задачи."""

```

```

        job_type = data.get("job_type")
        await asyncio.sleep(2)

        if job_type == "fatal_error":
            raise ValueError("Симуляция фатальной ошибки")

        return {"job_type": job_type, "status": "done", "processed_at": datetime.
utcnow().isoformat()}

    async def send_to_dlq(self, data: dict, error: str, retries: int):
        await self.producer.send(DLQ_TOPIC, {
            **data,
            "_dlq_meta": {
                "error": error,
                "retries": retries,
                "moved_at": datetime.utcnow().isoformat()
            }
        })

    async def run(self):
        await self.start()

        try:
            async for msg in self.consumer:
                if not self._running:
                    break

                msg_id = msg.value.get("job_id") or f"{msg.partition}-{msg.offse
t}"

                retries = msg.value.get("_retries", 0)

                logger.info(f"Получено: partition={msg.partition}, offset={msg.of
fset}, id={msg_id}")

                # Идемпотентность
                if await self.is_duplicate(msg_id):
                    logger.info(f"Дубль {msg_id}, пропускаю.")
                    await self.consumer.commit()
                    continue

                try:
                    result = await self.process(msg.value)
                    await self.mark_processed(msg_id)
                    await self.consumer.commit()
                    logger.info(f"Успех: {result}")

```

```

        except Exception as e:
            retries += 1
            logger.error(f"Ошибка обработки {msg_id} (попытка {retries}):
{e}")

            if retries >= MAX_RETRIES:
                await self.send_to_dlq(msg.value, str(e), retries)
                await self.consumer.commit()
                logger.warning(f"Отправлено в DLQ: {msg_id}")
            else:
                # В реальном коде здесь можно использовать retry с задерж
                # Пока просто не коммитим — Kafka переотправит
                pass

        except asyncio.CancelledError:
            logger.info("Получен сигнал остановки.")
        finally:
            await self.stop()

    async def main():
        consumer = RobustConsumer()

        loop = asyncio.get_event_loop()
        for sig in (signal.SIGINT, signal.SIGTERM):
            loop.add_signal_handler(sig, lambda: asyncio.create_task(consumer.stop
            ()))

        await consumer.run()

if __name__ == "__main__":
    asyncio.run(main())

```

12.8. Итоги модуля и всего курса: чек-лист

По Модулю 12:

- ☐ Понимаю, что такое **горизонтальное масштабирование**: добавление экземпляров воркеров.
- ☐ Умею запускать несколько Celery Workers, которые делят очередь.
- ☐ Знаю, что в Kafka количество параллельных консьюмеров в группе ограничено числом **партиций**.

- ☐ Понимаю, что такое **ребалансировка**: перераспределение партиций при изменении состава группы.
- ☐ Знаю, что ребалансировка инициируется при добавлении, удалении или «смерти» консьюмера.
- ☐ Знаю разницу между **Eager** (все сбрасывают партиции) и **Cooperative** (перемещаются только нужные) ребалансировкой.
- ☐ Увидел ребалансировку вживую, запуская и убивая Python-консьюмеров.
- ☐ Понимаю, почему сообщения могут обрабатываться **дважды** (at-least-once, падение до commit, ребалансировка).
- ☐ Знаю определение **идемпотентности**: повторный вызов не меняет результата.
- ☐ Умею реализовывать идемпотентность через **уникальный message_id** и проверку в Redis/БД.
- ☐ Понимаю, что проверка должна быть **атомарной** с бизнес-операцией (или близкой к этому).
- ☐ Знаю паттерн **Dead Letter Queue (DLQ)** для изоляции «битых» сообщений.
- ☐ Умею отправлять сообщение в DLQ после исчерпания retry.
- ☐ Собрал **устойчивый консьюмер**: с идемпотентностью, ручным commit, DLQ и graceful shutdown.

По всему курсу:

- ☐ Умею создавать **Docker-образы** с оптимальным порядком слоёв и кэшированием.
- ☐ Применяю **multi-stage builds** для уменьшения размера ML-контейнеров.
- ☐ Пишу **docker-compose.yml** для связки API, БД и брокера с **volumes**.
- ☐ Понимаю паттерн **Producer/Consumer** и разницу между эфемерными задачами и персистентными событиями.
- ☐ Работаю с **Redis Streams** (`XADD` , `XREAD` , группы, ACK).
- ☐ Организую очереди задач через **Celery + Redis** с Result Backend.
- ☐ Разворачиваю **Apache Kafka** (KRaft), создаю топики, партиции, работаю с консольными утилитами.
- ☐ Использую **aiokafka** для асинхронной интеграции Kafka с FastAPI.
- ☐ Масштабирую воркеров, понимаю **ребалансировку** и реализую **идемпотентность**.
- ☐ Готов к построению production-ready ML-инфраструктуры.